

SEMINAR · KIỂM THỬ PHẦN MỀM

Automation for Web

Khảo sát & thực nghiệm công cụ kiểm thử tự động cho ứng dụng web

Nhóm 5:

Nguyễn Thành Dâng - 23127334

Lê Nhựt Duy - 23127178

Trương Thành Đạt - 23127344

Ngô Thế Đạt - 23127340

Nguyễn Huy Quân - 23127107

AGENDA

Nội dung trình bày

- Lý thuyết: Automation for Web là gì, tại sao cần, khi nào dùng
- Nguyên lý hoạt động & tiêu chí đánh giá công cụ
- Khảo sát tổng quan 5 công cụ + hướng MCP
- Đào sâu 2 công cụ chính: Selenium & Playwright (kèm AI/MCP)
- So sánh trực tiếp Selenium ↔ Playwright
- Kết luận, khuyến nghị & Q&A

Automation testing for web là gì?

Dùng công cụ phần mềm để tự động thực thi các ca kiểm thử trên trình duyệt — mô phỏng thao tác người dùng (mở trang, nhập liệu, nhấp, điều hướng) rồi so kết quả thực tế với kết quả mong đợi.

Manual vs Automation

- Manual: con người thao tác — chậm, dễ sai, khó lặp lại
- Automation: script chạy tự động — nhanh, nhất quán, lặp vô hạn
- Automation không thay thế hoàn toàn manual mà bổ trợ

Tại sao cần?

Tốc độ & lặp lại

- Chạy lại hàng trăm ca trong vài phút
- Hợp kiểm thử hồi quy sau mỗi thay đổi



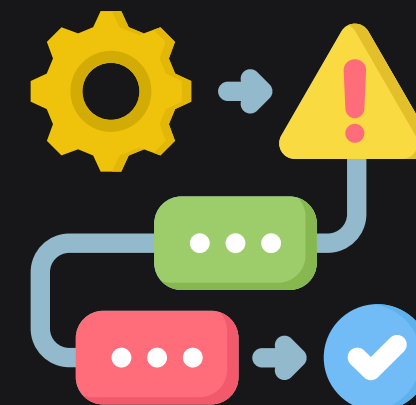
Chất lượng

- Giảm lỗi do con người
- Kết quả nhất quán, có báo cáo
- Cross-browser: Chrome/Firefox/Safari



Quy trình

- Tích hợp CI/CD, chạy tự động khi build
- Phát hiện lỗi sớm, tiết kiệm chi phí sửa
- Giải phóng tester cho test khám phá



Khi nào nên / không nên dùng?

NÊN dùng

- Ca kiểm thử lặp lại nhiều lần (hồi quy)
- Luồng nghiệp vụ ổn định, giá trị cao
- Cần chạy trên nhiều trình duyệt/nền tảng
- Kiểm thử smoke/sanity trong CI/CD

KHÔNG nên (ưu tiên manual)

- Giao diện thay đổi liên tục
- Test chỉ chạy một lần
- Đánh giá thẩm mỹ / trải nghiệm (UX)
- Khám phá (exploratory) đòi hỏi trực giác

Nguyên lý hoạt động chung

Hai cơ chế điều khiển trình duyệt

- WebDriver (chuẩn W3C): gửi lệnh tới driver — cách của Selenium
- DevTools / driver riêng: giao tiếp thẳng engine — Playwright, Cypress (nhANH, auto-wait)

Định vị phần tử (locator)

- id · css · xpath · text/role — nền tảng để tương tác chính xác

Mẫu một ca kiểm thử

- Arrange — chuẩn bị dữ liệu, mở trang
- Act — thực hiện thao tác (click, type...)
- Assert — khẳng định kết quả mong đợi

Tiêu chí đánh giá công cụ

1

Chức năng

Phạm vi tự động hoá, tính năng nổi bật

2

Giá

Mã nguồn mở/miễn phí hay thương mại

3

Điểm mạnh

Lợi thế cạnh tranh

4

Điểm yếu

Hạn chế cần lưu ý

5

Ngôn ngữ

Ngôn ngữ lập trình hỗ trợ

6

Hỗ trợ AI

Sinh script, self-healing, agent

Năm công cụ tiêu biểu

Selenium

Chuẩn kinh điển, WebDriver W3C, đa ngôn ngữ/trình duyệt

Playwright

Hiện đại, auto-wait, cross-browser, gỡ lỗi mạnh

Cypress

DX tốt, time-travel, hợp front-end JS

Katalon Studio

Low-code, all-in-one, AI (StudioAssist)

Robot Framework

Keyword-driven dễ đọc, nền Playwright

Bảng so sánh tổng hợp

Công cụ	Giá	Ngôn ngữ	AI	Phù hợp nhất
Selenium	Miễn phí	Java, Py, C#, JS...	Không	Đa trình duyệt, chuẩn W3C
Playwright	Miễn phí	JS/TS, Py, Java, C#	Có	Dự án mới, cross-browser
Cypress	Free + Cloud	JS/TS	Hạn chế	Front-end JS (React/Vue)
Katalon	Free + trả phí	Low-code (Groovy)	Mạnh	Đội ít code, low-code+AI
Robot FW	Miễn phí	Keyword (Py/JS)	Không	Kiểm thử chấp nhận

Selenium — Tổng quan & kiến trúc

- Bộ công cụ mã nguồn mở lâu đời, phổ biến nhất — chuẩn W3C WebDriver
- Hệ sinh thái: WebDriver + Selenium IDE + Selenium Grid
- Điều khiển trình duyệt thật qua driver (ChromeDriver, GeckoDriver...)
- Grid: chạy song song, phân tán trên nhiều máy/trình duyệt



Selenium — Tính năng & đánh giá

Ưu điểm

- Cộng đồng & tài liệu lớn nhất
- Hỗ trợ nhiều ngôn ngữ & trình duyệt nhất
- Chuẩn W3C, tích hợp CI/CD + Grid

Nhược điểm

- Cú pháp dài dòng
- Không auto-wait → dễ flaky
- Cài đặt nhiều, tốc độ chậm hơn

Ngôn ngữ

- Java, Python, C#, JavaScript, Ruby, Kotlin

Khi nào chọn?

- Cần đa trình duyệt/nền tảng rộng, đa ngôn ngữ, chuẩn W3C

Selenium — Code & AI/MCP

```
driver.get(BASE)
driver.find_element(By.CSS_SELECTOR,
    "input[type='email']").send_keys(user)

driver.find_element(By.CSS_SELECTOR,
    "input[type='password']").send_keys(pw)

driver.find_element(By.XPATH,
    "//button[contains(., 'Log in')]").click()

# phải tu WebDriverWait de tranh flaky
wait.until(EC.url_contains('inventory'))
```

AI / MCP

- Không có AI/MCP tích hợp sẵn
- Self-healing qua bên thứ ba (Healenium)
- Vẫn có thể để AI agent điều khiển Selenium qua MCP server cộng đồng
- Điểm yếu chính: locator cứng dễ vỡ khi UI đổi

Playwright — Tổng quan

Kiến trúc Native. Giao tiếp trực tiếp Browser Engine qua WebSocket — loại bỏ độ trễ của tầng WebDriver trung gian.

Unified API. Viết script một lần, chạy mượt trên Chromium, WebKit (Safari) và Firefox.

Auto-wait cốt lõi. Tự chờ element đạt trạng thái "actionable" (visible, enabled) trước khi thao tác — triệt tiêu flaky test do độ trễ mạng / animation UI.



Playwright — Tính năng & đánh giá

Tính năng & ưu điểm

- Codegen: sinh script từ thao tác
- Trace viewer + report HTML sẵn
- Cross-browser thật (gồm WebKit/Safari)
- Auto-wait, chạy song song, tốc độ cao
- Đa ngôn ngữ với tính năng tương đương

Nhược điểm

- Cộng đồng trẻ hơn Selenium
- Không hỗ trợ trình duyệt quá cũ
- Async/await gây bối ngỡ ban đầu

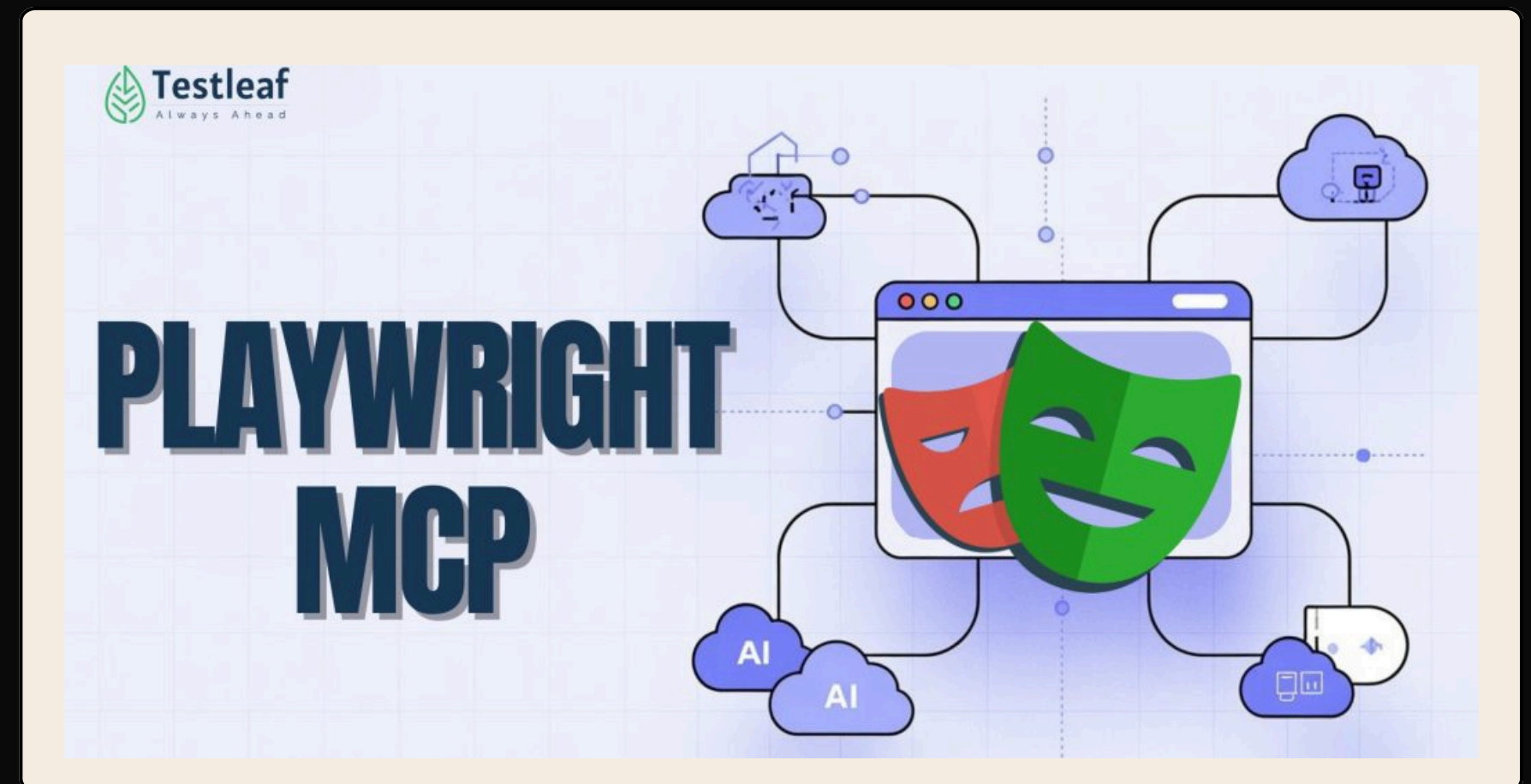
Playwright — Code & MCP

CÓ MCP — khép kín 1 phiên chat

- Mở trang thật — browser_navigate
- Đọc Accessibility Tree thật — browser_snapshot
- Suy ra selector đúng (getByRole / getByLabel)
- Thao tác + tự Verify → phát hiện bug ngay
- Chốt login.spec.ts, selector đã xác nhận

Vấn đề — Agent bị "Mù" (không MCP)

- Chỉ phân tích DOM từ mã nguồn tĩnh (jsx) → dễ "ảo giác" (hallucinate) locator
- Hiểu sai UI động: nhầm Placeholder với Label
- Test hay fail → Dev/QA tự mở DevTools (F12) vá thủ công



SO SÁNH TRỰC TIẾP

Selenium ↔ Playwright

Tiêu chí	Selenium	Playwright
Cơ chế chờ	Tự viết explicit wait	Auto-wait sẵn có
Số dòng code	Dài hơn	Ngắn gọn hơn
Độ ổn định	Dễ flaky	Ít flaky
Tốc độ	Chậm hơn	Nhanh hơn
Gỡ lỗi	Cần cấu hình thêm	Trace viewer + report sẵn
Trình duyệt	Rộng nhất (cả cũ)	Chromium/Firefox/WebKit
AI / MCP	Qua bên thứ ba	Playwright MCP (native)

Khuyến nghị chọn công cụ

Chốt lại

- Không có công cụ 'tốt nhất' tuyệt đối
- Phụ thuộc ngôn ngữ, phạm vi trình duyệt, ngân sách
- Playwright: mặc định tốt cho dự án mới
- Selenium: khi cần phạm vi rộng & chuẩn W3C

Chọn theo bối cảnh

- Dự án mới, cross-browser → Playwright
- Thuần JS/TS, DX → Cypress
- Phạm vi rộng, đa ngôn ngữ → Selenium
- Đội ít code + AI → Katalon
- Ca dễ đọc theo keyword → Robot FW
- Khám phá bằng AI → Playwright MCP

Tài liệu tham khảo

- Selenium: <https://www.selenium.dev/documentation/>
- Playwright: <https://playwright.dev/docs/intro>
- Cypress: <https://docs.cypress.io/>
- Katalon Studio: <https://docs.katalon.com/>
- Robot Framework: <https://docs.robotframework.org/>
- Playwright MCP: <https://playwright.dev/mcp/introduction>

Q&A

Cảm ơn thầy cô và các bạn đã lắng nghe!

Automation for Web · Seminar Kiểm thử phần mềm